Regularizer in DNN

Gwangsu Kim

JBNU Dep. of Stat.

First Semester 2025

Special Topics for Machine Learning

모형 복잡하게 -> overfitting??

Outline

Intro.

Regularizarion with Norm constraints and Others

Early stopping

Parameter Tying and Sharing, CNN & sparsity

Ensembles and Dropout

Intro. I

1. Intro.

- An effective regularizer is one that makes a profitable trade, reducing variance significantly while not overly increasing the bias.
- Three situations: (1) excluded the true data-generating process—corresponding to underfitting and inducing bias, (2) matched the true data-generating process, or (3) included the generating process but also many other possible generating processes.
- Focusing on the case of (3).

Intro. II

2. Parameter Norm Penalties

- Restriction to the norm of parameters.

$$J(\boldsymbol{X}, \boldsymbol{y} \mid \boldsymbol{w}) + \alpha\Omega(\boldsymbol{w}), \alpha \geq 0$$

- Weights and bias: conventionally, we put a restriction on weights.
- L_2 regularization

$$J(\boldsymbol{X}, \boldsymbol{y} \mid \boldsymbol{w}) + \frac{\alpha}{2}\boldsymbol{w}^\intercal \boldsymbol{w} \tag{1}$$

$$\begin{aligned} \boldsymbol{w} &\leftarrow \boldsymbol{w} - \epsilon(\alpha\boldsymbol{w} + \nabla_{\boldsymbol{w}} J(\boldsymbol{X}, \boldsymbol{y} \mid \boldsymbol{w})) \\ &= (1 - \epsilon\alpha)\boldsymbol{w} - \epsilon\nabla_{\boldsymbol{w}} J(\boldsymbol{X}, \boldsymbol{y} \mid \boldsymbol{w})) \end{aligned}$$

1. When $\boldsymbol{w}^* = \text{argmin}_{\boldsymbol{w}} J(\boldsymbol{X}, \boldsymbol{y} \mid \boldsymbol{w})$,

Intro. III

$$
\begin{aligned}
& J(\boldsymbol{w}) \\
= \quad & J(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^*)^\intercal H (\boldsymbol{w} - \boldsymbol{w}^*)
\end{aligned}
$$

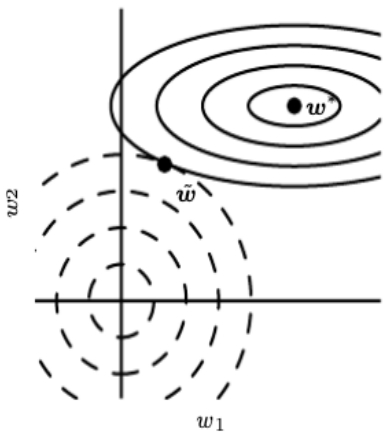
inducing the following:

$$
0 = \nabla_{\hat{\boldsymbol{w}}} J(\boldsymbol{w}) = H(\hat{\boldsymbol{w}} - \boldsymbol{w}^*)
$$

where $\hat{\boldsymbol{w}}$ is the minimum of J. Adding the $\frac{\alpha}{2}\boldsymbol{w}^\intercal \boldsymbol{w}$ to $J(\boldsymbol{w})$, we have

$$
\begin{aligned}
0 \quad &= \quad \alpha\tilde{\boldsymbol{w}} + H(\tilde{\boldsymbol{w}} - \boldsymbol{w}^*) \\
\tilde{\boldsymbol{w}} \quad &= \quad Q(\Lambda + \alpha \boldsymbol{I})^{-1} \Lambda Q^\intercal \boldsymbol{w}^*
\end{aligned}
$$

where $H = Q\Lambda Q^\intercal$.



Intro. IV

2. L_1 regularization

$$\tilde{J}(\boldsymbol{w}) = J(\boldsymbol{w}) + \alpha \|\boldsymbol{w}\|_1$$

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}) = \alpha \text{sign}(\boldsymbol{w}) + H(\boldsymbol{w} - \boldsymbol{w}^*)$$

For simplicity, we assume that the H is a diagonal matrix from $\{H_{ii}\}$, then

$$\tilde{w}_i = \text{sgn}(w_i^*) \max \left\{ |w_i^*| - \frac{\alpha}{H_{ii}}, 0 \right\}.$$

$*$ It implies that the smaller becomes 0 and a shift occurs for non-zeros (variable selection property).

3. Closely related to Bayesian MAP (maximum a posterior) with the Laplacian priors for $\boldsymbol{w}$.

Regularizarion with Norm constraints and others I

1. Norm constraints with explicit way
   - Lagrangian

$$
\begin{aligned}
L(\boldsymbol{w}, \alpha) &= J(\boldsymbol{w}) + \alpha(\Omega(\boldsymbol{w}) - k) \\
\boldsymbol{w}^* &= \arg\min_{\boldsymbol{w}} \max_{\alpha, \alpha \geq 0} L(\boldsymbol{w}, \alpha)
\end{aligned}
$$

   - We can control α but cannot know the k exactly (increasing α makes the Lag. term larger when $\Omega(\boldsymbol{w}) > k$, but arg min term can be larger).

Regularizarion with Norm constraints and others II

- In some cases, we can use k not α by the projection onto $\Omega(\boldsymbol{w}) < k$.
  1. Explicit constraints implemented by reprojection have an effect only when the weights become large and attempt to leave the constraint region.
  2. Some stability in optimization when used in a layer-wise manner. Total norm cannot avoid some weight inflating.

Regularizarion with Norm constraints and others III

2. Others

- Dataset augmentation: create fake data and add it to the training set.
  1. Various transformations such as flip not changing the label.
  2. Injecting the noise in the input (multi-layer de-nosing).
  3. Be careful to validate the performance of algorithms due to the data augmentations.
- Noise robustness
  1. Considering

$$J = \mathbb{E}_{p(\boldsymbol{x},y)}[(\hat{y}(\boldsymbol{x} \mid \boldsymbol{w}) - y)^2]$$

.

  2. Noise perturbation: $\epsilon_{\boldsymbol{w}} \sim \mathcal{N}(\epsilon \mid \boldsymbol{0}, \eta \boldsymbol{I})$

$$J_{\epsilon_{\boldsymbol{w}}} = \mathbb{E}_{p(x,y,\epsilon_{\boldsymbol{w}})}[(\hat{y}(\boldsymbol{x} \mid \boldsymbol{w} + \epsilon_{\boldsymbol{w}}) - y)^2]$$

  3. It results in the penalized term as $\eta \mathbb{E}_{p(\boldsymbol{x},y)} \left[\|\nabla_{\boldsymbol{w}} \hat{y}(\boldsymbol{x} \mid \boldsymbol{w})\|^2 \right]$.

Regularizarion with Norm constraints and others IV

4. What's the effect of perturbation: robustness with respect to variation of weights (plateau).

- Input noisy to target: target smoothing (example)

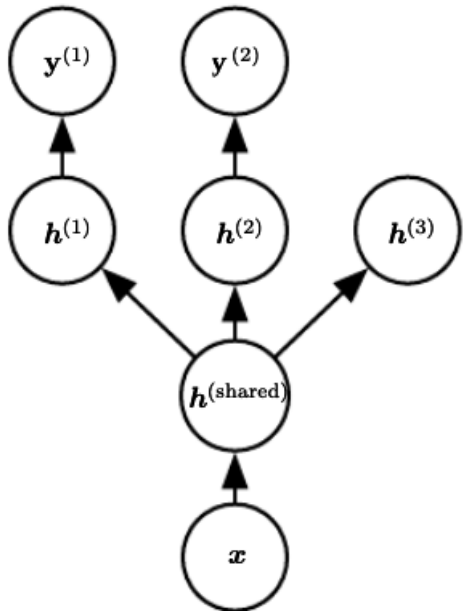
$$(0, 1) \to (\epsilon, 1 - \epsilon)$$

where $\epsilon > 0$ is small.

Regularizarion with Norm constraints and others V

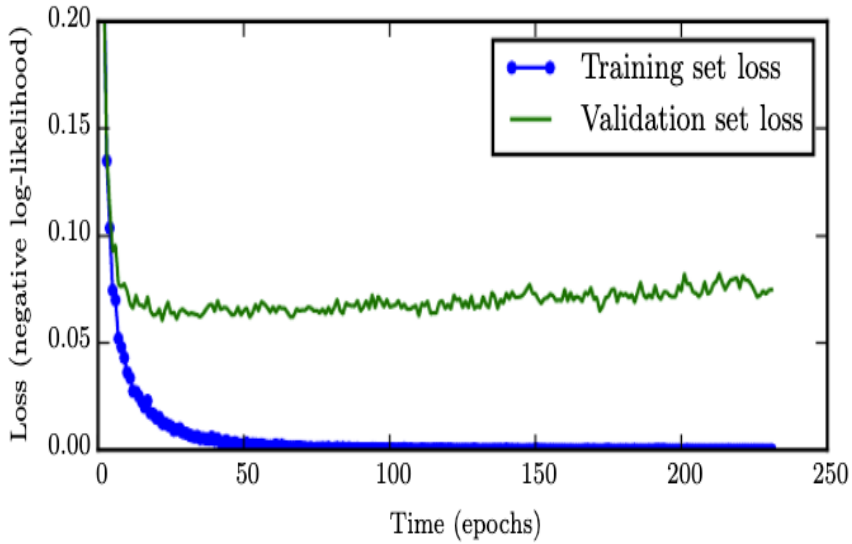
- Semi-supervised learning: we have data $\boldsymbol{x}$ and $(\boldsymbol{x}, y)$ in partial.
  1. Can the sole $\boldsymbol{x}$ can be helpful for the prediction of $p(y \mid \boldsymbol{x})$?
  2. Yes: shared weights for $p(\boldsymbol{x})$ and $p(y \mid \boldsymbol{x})$ in learning.
- Multi-task learning
  1. Scenarios: multi-tasks such as classifications and regressions, and feature extractors are almost the same exception for the latter layers.
  2. Can we use the first parts for the various tasks? (yes, it can be very helpful).

Regularizarion with Norm constraints and others VI



Early stopping I

1. Strategy to find the best epochs
   - Observing the validation error (loss) and choosing the hyperparameter for epochs.



Early stopping II

- The early stopping meta-algorithm.
  1. When the validation error after n training steps decreases, we choose the epoch number n. If not, initialize and start to validate the above (set the upper limit for this trial).
  2. This procedure requires weights at each trial.
  3. It is an unobtrusive form of regularization and cooperates with other regularization approaches.
- A meta-algorithm for using early stopping to determine how long to train.

  • Divide the training dataset into two parts. One is for determining the number of n. The other is for training.

Early stopping III

- Meta-algorithm using early stopping to determine at what objective value we start to overfit.

  • Divide the training dataset into two parts. Calculation of loss by the training on one part, and save it as ϵ.

  • Training procedure uses the other part until the (validation) loss is greater than the ϵ.

Early stopping IV

- Comparing the early stopping with L_2 regularization.

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}) = H(\boldsymbol{w} - \boldsymbol{w}^*)$$

$$\begin{aligned} \boldsymbol{w}^{(\tau)} - \boldsymbol{w}^* &= \boldsymbol{w}^{(\tau-1)} - \epsilon \nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}) \\ &= (\boldsymbol{I} - \epsilon H)(\boldsymbol{w}^{(\tau-1)} - \boldsymbol{w}^*) \\ Q^\intercal(\boldsymbol{w}^{(\tau)} - \boldsymbol{w}^*) &= (\boldsymbol{I} - \epsilon \Lambda) Q^\intercal(\boldsymbol{w}^{(\tau-1)} - \boldsymbol{w}^*) \end{aligned}$$

By updating with small $\epsilon > 0$,

$$Q^\intercal \tilde{\boldsymbol{w}} = \left[\boldsymbol{I} - (\boldsymbol{I} - \epsilon \Lambda)^\tau\right] Q^\intercal \boldsymbol{w}^*$$

where $\boldsymbol{w}^{(0)} = 0$. Note that L_2 regularization implies

$$Q^\intercal \tilde{\boldsymbol{w}} = (\boldsymbol{I} - (\Lambda + \alpha \boldsymbol{I})^{-1} \alpha) Q^\intercal \boldsymbol{w}^*.$$

Early stopping V

- Interestingly, the following hols

$$(\boldsymbol{I} - \epsilon\Lambda)^{\tau} \approx (\Lambda + \alpha\boldsymbol{I})^{-1}\alpha$$

  where $\tau \approx \frac{1}{\epsilon\alpha}$ and λ_is are small.

- Message: early stopping is related to L_2 regularization.

Parameter Tying and Sharing, CNN, & Sparsity I

1. Parameter Tying and Sharing

- Tying: We often want two parameters to become similar.

$$f(\boldsymbol{w}_A, \boldsymbol{x}) \text{ and } f(\boldsymbol{w}_B, \boldsymbol{x})$$

$$\alpha \|\boldsymbol{w}_A - \boldsymbol{w}_B\|$$

- Sharing: $\boldsymbol{w}_A = \boldsymbol{w}_B$.

Parameter Tying and Sharing, CNN, & Sparsity II

2. CNN
   - CNN uses weight sharing and sparse connection between layers.
   - Details are explained later.
3. Sparsity
   - Penalty term

$$\tilde{J}(\boldsymbol{w}) = J(\boldsymbol{w}) + \alpha\Omega(\boldsymbol{h}),$$

     where $\boldsymbol{h}$ is a vector of hidden nodes.
   - Orthogonal matching pursuit

$$\arg\min_{\boldsymbol{h}, \|\boldsymbol{h}\|_0 < k} \|\boldsymbol{x} - \boldsymbol{W}\boldsymbol{h}\|_2^2$$

Ensembles and Dropout I

1. Model Averaging

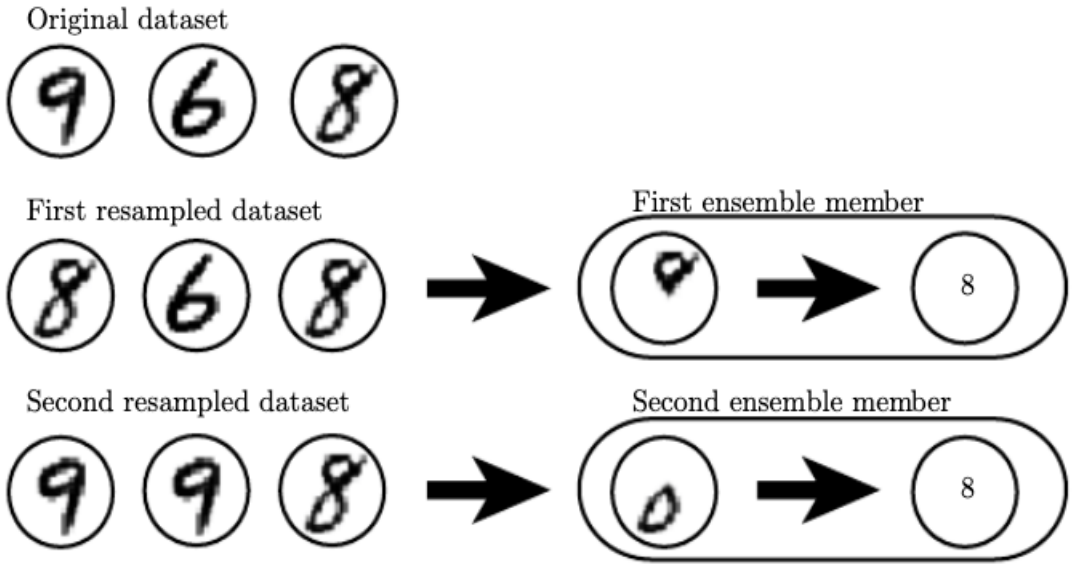
- Assume that ϵ_i is the error term for the i-th model.

$$\begin{aligned}\mathbb{E}\left[\left(\frac{1}{k}\sum_{i=1}^{k}\epsilon_i\right)^2\right] &= \frac{1}{k^2}\mathbb{E}\left[\sum_i \epsilon_i^2 + \sum_{i\neq j}\epsilon_i\epsilon_j\right] \\ &= \frac{v}{k} + \frac{k-1}{k}c\end{aligned}$$

  where $\mathbb{E}[\epsilon_i] = 0, \mathbb{E}[\epsilon_i^2] = v$ and $\mathbb{E}[\epsilon_i\epsilon_j] = c$ $(i \neq j)$

- It implies that the lower c is required for the ensembles for high performance.

Ensembles and Dropout II

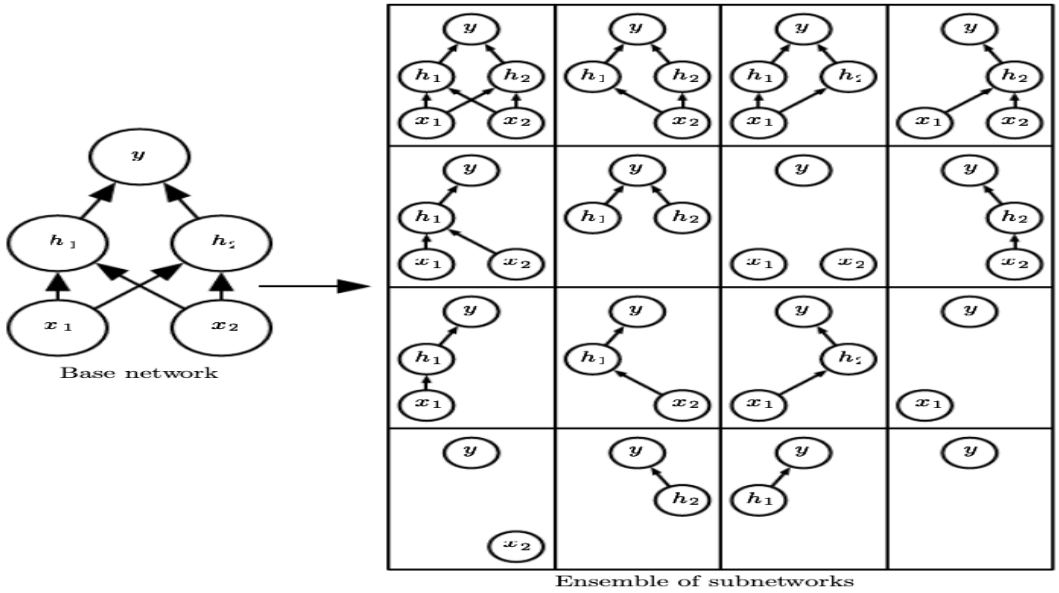


Bagging example by bootstrap.

2. Dropout

- Most famous approaches.
- dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from an underlying base network.

Ensembles and Dropout III



Bagging example by bootstrap.

- Some networks look non-functional. When the number of nodes increases the non-functional cases appear in little probability.
- Conceptually, we should make many sub-networks and learning is done for each sub-network (too expensive!!).

Ensembles and Dropout IV

- Cooperated with the stochastic gradient, using random sub-networks at each gradient update.
- Note
  1. Independence is not ensured. In fact, this approach is based on the correlated sub-networks.
  2. Aggregated prediction.

$$\frac{1}{k}\sum_{i} p^{(i)}(y \mid \boldsymbol{x})$$

  3. In practice, we use various sub-networks through UPDATES. Therefore, for each parameter θ_k, there are different l_k updates.
  4. We use $\boldsymbol{\mu} \in \{0,1\}^d$, and $\boldsymbol{\mu} \odot \boldsymbol{\theta}$ where $\boldsymbol{\theta}$ is a vector stacked from $\boldsymbol{W}$ and $\boldsymbol{b}$.

Ensembles and Dropout V

5. Geometric mean instead of arithmetic mean.

$$\tilde{p}_{ens}(y \mid \boldsymbol{x}) = \left\{ \prod_{\boldsymbol{\mu}} p(y \mid \boldsymbol{x}, \boldsymbol{\mu}) \right\}^{1/2^d}$$

where 2^d is number of sub-networks (on-off d connections).

$$p_{ens}(y \mid \boldsymbol{x}) = \frac{\tilde{p}_{ens}(y \mid \boldsymbol{x})}{\sum_{y'} \tilde{p}_{ens}(y' \mid \boldsymbol{x})}$$

Ensembles and Dropout VI

- **Weight scaling inference rule.**

  Thorough weighting is not easy due to large sub-networks. Using the scaled weight parameters determined by the inclusion (connection) probability p.

$$\begin{aligned} \mathbb{P}(y = c \mid v) &= \text{softmax}(\boldsymbol{W}^{\mathsf{T}} \boldsymbol{v} + \boldsymbol{b})_c \\ &\quad \text{softmax}(\boldsymbol{W}^{\mathsf{T}} (\boldsymbol{d} \odot \boldsymbol{v}) + \boldsymbol{b})_c \end{aligned}$$

  where $\boldsymbol{d}$ is binary.

Ensembles and Dropout VII

$$\mathbb{P}(y = c \mid \boldsymbol{v}) \quad = \quad \frac{\tilde{\mathbb{P}}_{ens}(y \mid \boldsymbol{v})}{\sum_{y'} \tilde{\mathbb{P}}_{ens}(y' \mid \boldsymbol{v})}$$

where $\tilde{\mathbb{P}}_{ens}(y \mid \boldsymbol{v}) = \left\{ \prod_{\boldsymbol{d} \in \{0,1\}^n} \mathbb{P}(y \mid \boldsymbol{v}, \boldsymbol{d}) \right\}^{1/2^n}$.

- Here, we have

$$\begin{aligned} \tilde{\mathbb{P}}_{ens}(y \mid \boldsymbol{v}) \quad &= \quad \left\{ \prod_{\boldsymbol{d} \in \{0,1\}^n} \mathbb{P}(y \mid \boldsymbol{v}, \boldsymbol{d}) \right\}^{1/2^n} \\ &= \quad \left\{ \prod_{\boldsymbol{d} \in \{0,1\}^n} \text{softmax}(\boldsymbol{W}^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + \boldsymbol{b})_y \right\}^{1/2^n} \end{aligned}$$

Ensembles and Dropout VIII

- Conti.

$$
\begin{aligned}
&= \left\{ \prod_{\boldsymbol{d} \in \{0,1\}^n} \text{softmax}(\boldsymbol{W}^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + \boldsymbol{b})_y \right\}^{1/2^n} \\
&= \left\{ \prod_{\boldsymbol{d} \in \{0,1\}^n} \frac{\exp(\boldsymbol{W}_y^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + b_y)}{\sum_{y'} \exp(\boldsymbol{W}_{y'}^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + b_{y'})} \right\}^{1/2^n} \\
&= \frac{\left[\prod_{\boldsymbol{d} \in \{0,1\}^n} \exp(\boldsymbol{W}_y^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + b_y) \right]^{1/2^n}}{\left[\prod_{\boldsymbol{d} \in \{0,1\}^n} \sum_{y'} \exp(\boldsymbol{W}_{y'}^\intercal (\boldsymbol{d} \odot \boldsymbol{v}) + b_{y'}) \right]^{1/2^n}}
\end{aligned}
$$

Ensembles and Dropout IX

- Finally,

$$\begin{aligned}
\tilde{\mathbb{P}}_{ens}(y \mid \boldsymbol{v}) &\propto \left[\prod_{\boldsymbol{d}\in\{0,1\}^n} \exp(\boldsymbol{W}_y^\intercal(\boldsymbol{d}\odot\boldsymbol{v}) + b_y)\right]^{1/2^n} \\
&= \exp\left\{\frac{1}{2^n}\sum_{\boldsymbol{d}\in\{0,1\}^n} \boldsymbol{W}_y^\intercal(\boldsymbol{d}\odot\boldsymbol{v}) + b_y\right\} \\
&= \exp\left\{\frac{1}{2}\boldsymbol{W}_y^\intercal\boldsymbol{v} + b_y\right\}
\end{aligned}$$

- The weight scaling rule is only an approximation for deep models that have non-linearities. Though the approximation has not been theoretically characterized, it often works well empirically (superior to the conventional ensembles).

Ensembles and Dropout X

- More effective than other standard computationally inexpensive regularizers, such as weight decay, norm constraints, and sparse activity regularization.
- Dropout may also be combined with other forms of regularization to yield further improvement
- Pros and cons
  1. Pros: Cheap computation, flexible to various models.
  2. Cons: Large networks and longer iterations. The gain is relatively small when the data are very large.

Ensembles and Dropout XI

- In a small dataset, Bayesian DNN can work well. The stochasticity is not necessary to achieve the regularizing effect of dropout. It is also not sufficient (fast dropout: approximating the average by the deterministic way, dropout boosting: masking and eliminating the regularization effects)

- Motivating the stochastic approaches.
  1. Drop connect: Considering the connection between a single node and single unit instead of layer to layer.
  2. Stochastic pooling: Random CNN to be assigned for different spatial locations.
  3. Changing $\boldsymbol{\mu}$ as a conti. r.v. such as $\boldsymbol{\mu} \sim \mathcal{N}(\mathbf{1}, \boldsymbol{I})$.

Ensembles and Dropout XII

- Advanced interpretation of dropout.
  1. Ensemble of models that share hidden units (information is focused). This can improve the performance.
  2. Masking noise is applied to the hidden units.
  3. Multiplicative noise (random connection).